

University of Ruhuna

Faculty of Technology

Bachelor of Information and Communication Technology (BICT)

Level II - Semester I

ICT2132 - Object Oriented Programming Practicum

Software Requirements Specification (SRS)

for

Faculty Management System

Mini Project - Group 05 Submission

Date: 13 September 2026

Submitted by - TG/2024/2095 - Hasaranga K.H.M

TG/2024/2096 - Himansana N.V.S

TG/2024/2080 - Appuhami I.A.S.S

TG/2024/2127 - Dahanayaka D.S.R.N

1. Introduction

1.1 Purpose

This document describes the Software Requirements Specification (SRS) for the Faculty Management System, a desktop application being developed as the ICT2132 mini project. The purpose of this document is to describe what the system will do, who will use it, and the main features it must provide

1.2 Scope

The Faculty Management System is a JavaFX desktop application that will support the Faculty of Technology in managing academic and administrative information. The system covers:

- User profile management for Admin, Lecturer, Technical Officer and Undergraduate
- Course management
- Undergraduate (student) management, including repeat and batch-missed students
- Attendance management (theory and practical)
- Medical record management
- Marks management
- Eligibility checking (attendance and CA marks)
- Grade calculation
- SGPA and CGPA calculation
- Notices
- Timetables
- Course materials

1.3 Objectives

- Replace manual/paper-based faculty record keeping with a single desktop application.
- Provide role-based access so each user only sees what is relevant to them.
- Automatically calculate attendance percentages, eligibility, grades, SGPA and CGPA.
- Store all faculty data in a central MySQL database.
- Demonstrate core OOP concepts (classes, inheritance, encapsulation, abstraction, polymorphism) in a real application.

1.4 References

- ICT2132 - Object Oriented Programming Practicum, Mini Project specification, University of Ruhuna.
- UGC Commission Circular No. 12-2024 (grading scheme reference).

2. Overall Description

2.1 Product Perspective

The system is a standalone JavaFX desktop application that connects to a MySQL database using JDBC. It follows a simple layered structure:

JavaFX GUI (FXML + CSS) -> Service / Business Logic Layer -> DAO (Data Access) Layer -> JDBC -> MySQL Database

The GUI layer displays screens and collects user input. The service layer applies business rules, such as attendance percentage and GPA calculations. The DAO layer is responsible for reading and writing data to MySQL through JDBC.

2.2 User Classes and Characteristics

Administrator	Manages user accounts, courses, notices and timetables. Has the highest level of access.
Lecturer	Manages course materials and marks, and views student attendance, medical and result information.
Technical Officer	Manages attendance and medical records for students, and views notices and timetables.
Undergraduate	Views their own attendance, medical, results, GPA, timetable, notices and course materials.

2.3 Operating Environment

- Windows or Linux desktop computer
- Java JDK and JavaFX runtime
- MySQL Server (may be run locally or inside a Docker container)
- Development tools: IntelliJ IDEA, Maven, Git/GitHub

2.4 Design and Implementation Constraints

- The application must be built in Java using JavaFX.
- FXML is used for interface layout and CSS for styling.
- MySQL is used as the database, accessed through JDBC.
- Maven is used to manage project dependencies and the build.
- The system must follow OOP principles.
- Access to features depends on the logged-in user's role.

2.5 Assumptions and Dependencies

- Each user has one account with a single fixed role.
- The current semester timetable defines the subjects and credit values used for attendance.
- MySQL Server is available and reachable from the application.
- The application is used on a single computer or within a small local network, not over the public internet.

3. Functional Requirements

The main functional requirements are summarised below. Not every possible sub-feature is listed individually; this is a simplified requirement set sufficient for the mini project.

3.1 Login and Authentication

- FR-01: The system shall allow a user to log in with a username and password.
- FR-02: The system shall identify the user's role and load the matching dashboard.
- FR-03: The system shall reject invalid login attempts with an error message.
- FR-04: The system shall allow a logged-in user to log out.

3.2 User Profile Management

- FR-05: Admin shall be able to create, update, deactivate and assign roles to user accounts.
- FR-06: Lecturers and Technical Officers shall be able to update their own profile, except username and password.
- FR-07: Undergraduates shall be able to update only their contact details and profile picture.

3.3 Course Management

- FR-08: Admin shall be able to add, update and deactivate courses.
- FR-09: Lecturers shall be able to add and update materials for their assigned courses.
- FR-10: Students shall be able to view their enrolled/available courses.

3.4 Undergraduate Management

- FR-11: Admin shall be able to register and maintain undergraduate records, including batch, department, repeat and batch-missed status.

3.5 Attendance Management

- FR-12: Technical Officers shall be able to add and maintain theory and practical attendance records.
- FR-13: The system shall calculate attendance percentage separately for theory, practical, and combined.
- FR-14: The system shall support the attendance scenarios: more than 80%, exactly 80%, less than 80% without medical, more than 80% with medical, and less than 80% with medical.
- FR-15: Students shall be able to view their own attendance; Lecturers shall be able to view student attendance.

3.6 Medical Management

- FR-16: Technical Officers shall be able to add and update medical records and mark them as approved or not approved.
- FR-17: Students and Lecturers shall be able to view relevant medical information.

3.7 Marks Management

- FR-18: Lecturers shall be able to enter and update marks (out of 100) for different evaluation types.
- FR-19: Lecturers shall be able to view marks for an individual student or for the whole batch.

3.8 Eligibility Checking

- FR-20: The system shall determine exam eligibility based on CA marks (minimum 40%) and attendance.
- FR-21: The system shall show eligibility status for individual students and for a batch.

3.9 Grades and GPA

- FR-22: The system shall calculate final marks and assign grades according to the UGC grading scheme referenced in the project brief.
- FR-23: The system shall calculate SGPA and CGPA for each student, using subject credits and grades.

3.10 Notices, Timetables and Course Materials

- FR-24: Admin shall be able to create, update and delete notices; other roles shall be able to view relevant notices.
- FR-25: Admin shall be able to manage timetable entries; other roles shall be able to view relevant timetable information.
- FR-26: Lecturers shall be able to manage course materials; students shall be able to view them.

4. Non-Functional Requirements

NFR-01	The application should respond to normal user actions (login, viewing records) within a few seconds.
NFR-02	Passwords shall not be stored or displayed in plain text.
NFR-03	The interface should be simple and consistent so it is easy for staff and students to use.
NFR-04	The system should handle invalid input and database errors without crashing.
NFR-05	The codebase should be organised so it is reasonably easy to maintain and extend.
NFR-06	The application should run on standard Windows or Linux desktop computers.

5. Database Requirements (Overview)

The database will store the following main entities. A detailed ER diagram will be produced as a separate deliverable.

- User (base information shared by all roles)
- Student / Undergraduate
- Lecturer
- Technical Officer
- Course
- Attendance
- Medical
- Mark / Grade
- Notice
- Timetable
- Course Material

6. Object-Oriented Design Notes

Since this project is part of an OOP practicum, the design will demonstrate core OOP concepts:

- Classes and Objects: e.g. Student, Lecturer, Course, Attendance classes.
- Inheritance: Admin, Lecturer, TechnicalOfficer and Undergraduate can extend a common User class.
- Encapsulation: private fields with getters/setters in model classes.
- Abstraction: service/DAO interfaces hide implementation detail from the GUI layer.
- Polymorphism: e.g. a shared method such as viewDashboard() behaving differently per role.
- Exception Handling: Java try-catch used around database and input operations.

7. Use Cases (Summary)

Administrator	Login, Manage Users, Manage Courses, Manage Notices, Manage Timetables, Logout
Lecturer	Login, Update Profile, Manage Course Materials, Manage Marks, View Attendance/Medical, View Results, Logout
Technical Officer	Login, Update Profile, Manage Attendance, Manage Medical Records, View Notices/Timetable, Logout
Undergraduate	Login, Update Profile, View Courses/Attendance/Medical/Results/GPA/Timetable/Notices, Logout

8. Security Requirements

SEC-01	User passwords shall be stored using a one-way hashing algorithm; plaintext passwords shall never be persisted or logged.
SEC-02	Every system function shall check the authenticated user's role before executing, so that a user can only perform operations permitted for their role (role-based access control).
SEC-03	All user-supplied input shall be validated on the GUI layer and treated as untrusted by the Service and DAO layers.
SEC-04	All SQL statements shall use parameterised/prepared statements (via JDBC) to prevent SQL injection; string concatenation shall not be used to build queries containing user input.
SEC-05	Database access shall be restricted to a dedicated application database user with only the privileges required by the application, rather than a root/administrative MySQL account.
SEC-06	The system shall maintain the current user's session/state only for the duration of their login, clearing it fully on logout.
SEC-07	The system shall prevent unauthorised operations by re-validating permissions at the Service layer, not relying solely on hiding buttons in the GUI.
SEC-08	Database connection credentials shall be kept out of version-controlled source code (e.g., stored in a local configuration/properties file or environment variables excluded from Git) wherever practically possible for the project.

9. Future Enhancements

The following are possible future improvements and are not part of the current project scope:

- Mobile companion application
- Automated email/SMS notifications
- Cloud-hosted database
- Advanced analytics and reporting

10. Conclusion

This SRS outlines the essential requirements for the Faculty Management System to be developed using Java, JavaFX and MySQL as part of the ICT2132 mini project. It covers the core modules required by the assignment brief - user management, courses, attendance, medicals, marks, eligibility, grades, SGPA/CGPA, notices, timetables and course materials - while keeping the scope realistic for a Level II student group project.